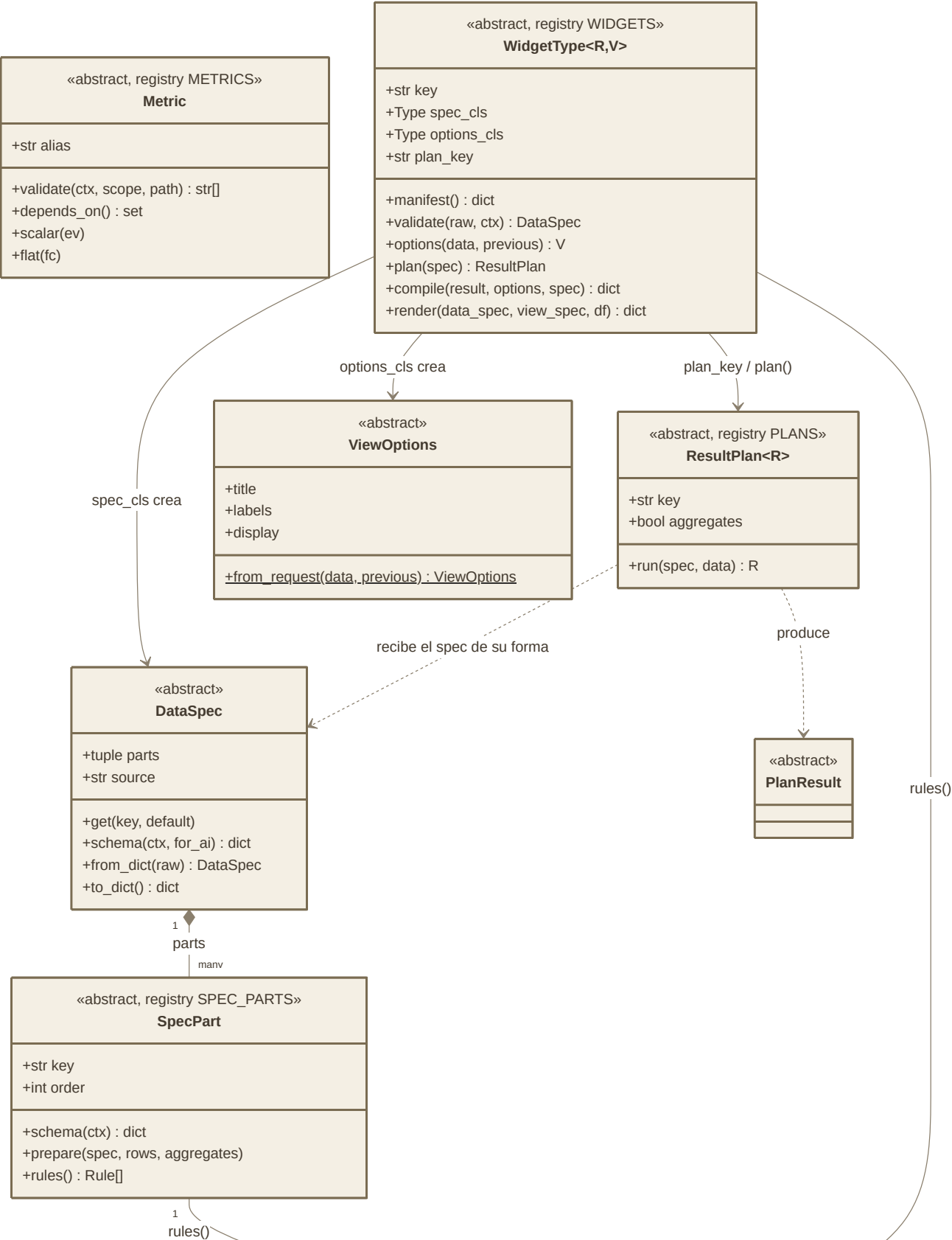
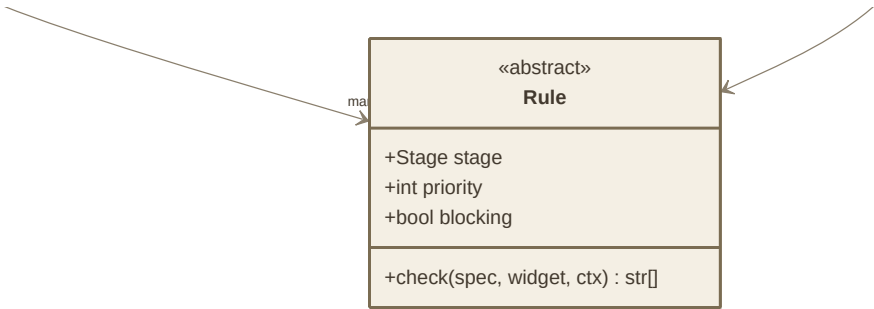


Diagramas de clases — arquitectura de widgets

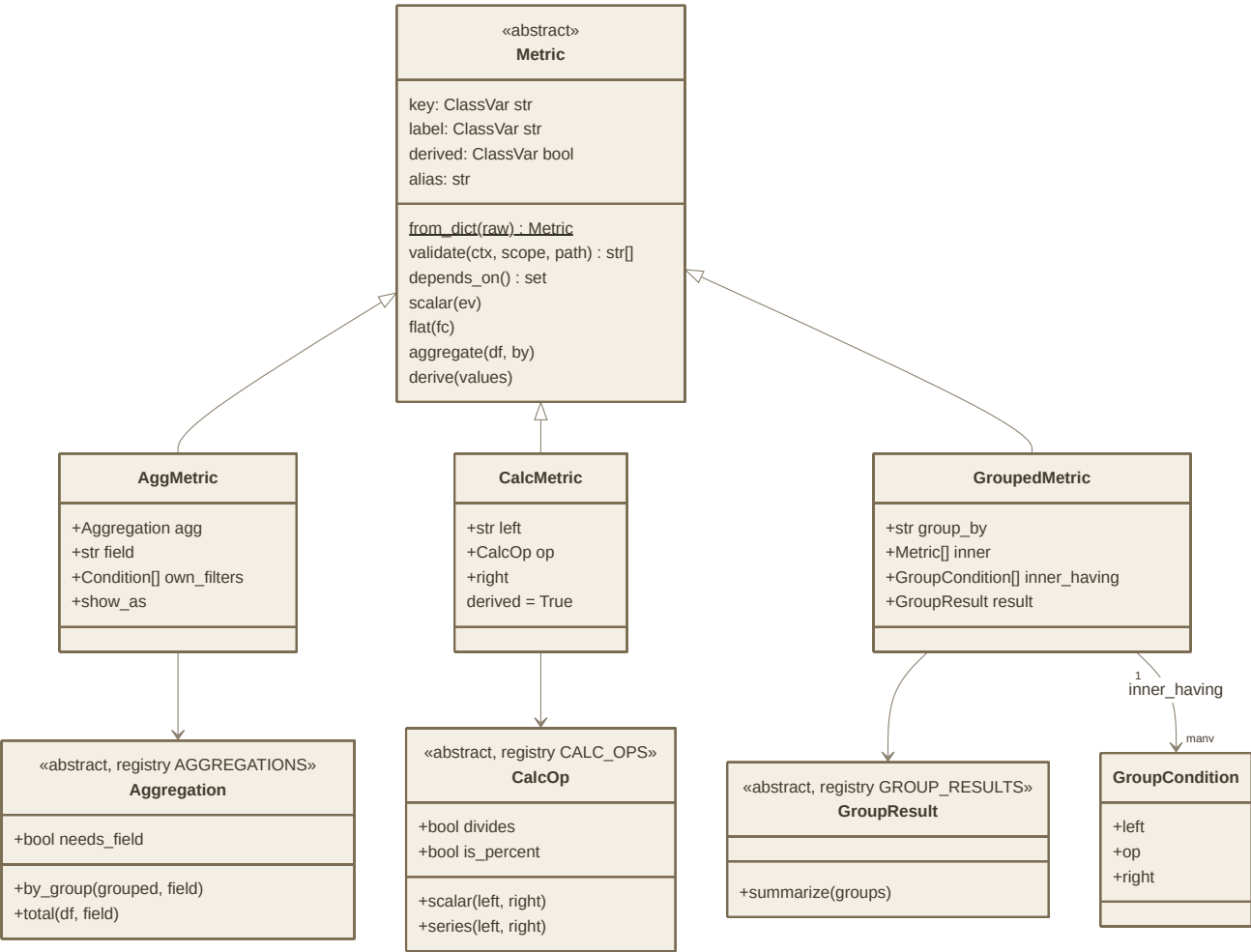
Un solo diagrama con las ~50 clases del sistema sería ilegible, así que está separado por zona, siguiendo el mismo orden que `arquitectura_widgets.md`. El primero es el panorama (las piezas centrales y cómo se conectan); los siguientes abren cada zona; el último junta todas las piezas en un caso real (el widget de histograma).

1 · Panorama — las piezas centrales



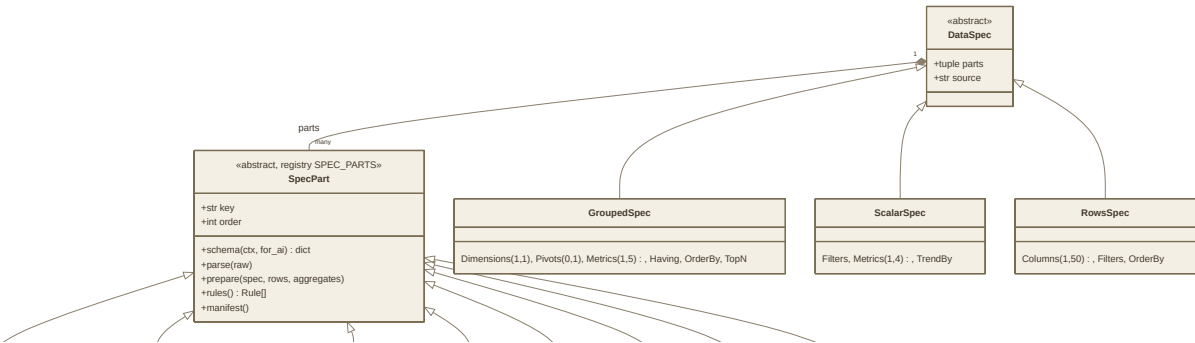


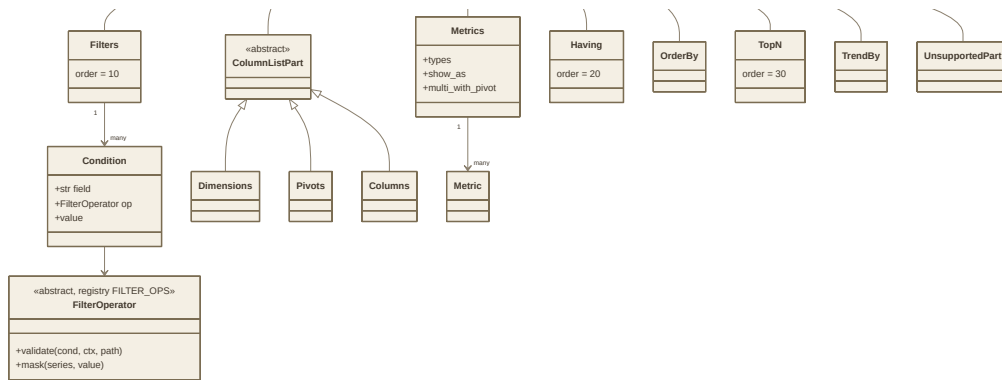
2 · dsl/ — Métricas



Nota: `evaluation_order(metrics)` ordena por `depends_on()` , no por posición en la lista — por eso arrastrar un `CalcMetric` por encima de lo que usa no lo rompe.

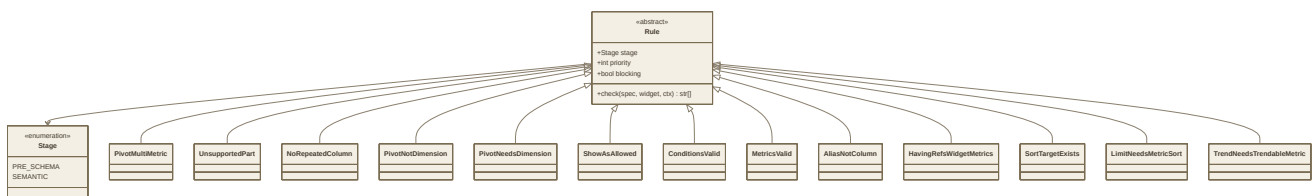
3 · dsl/ — DataSpec y sus piezas





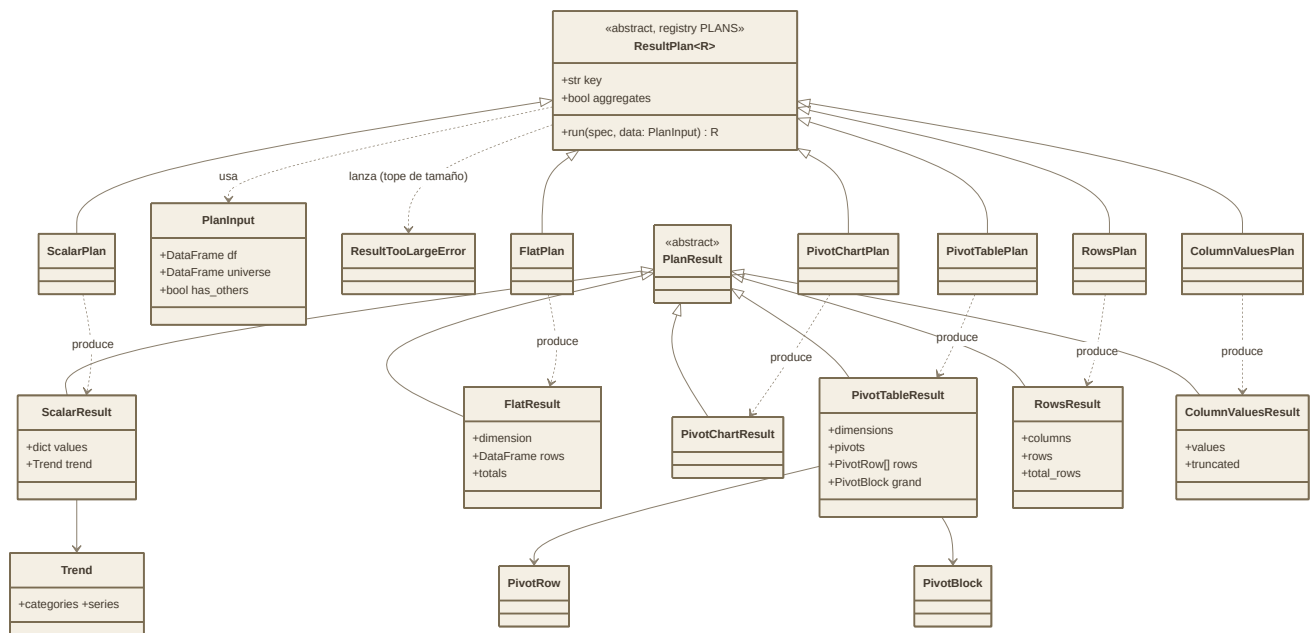
GroupedSpec / ScalarSpec / RowsSpec *nacen de* with_parts(Base, *piezas, without=...) — *un widget ajusta cotas reemplazando piezas, no heredando en cadena.*

4 · dsl/ — Reglas semánticas

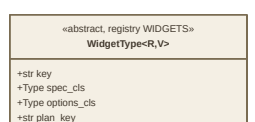


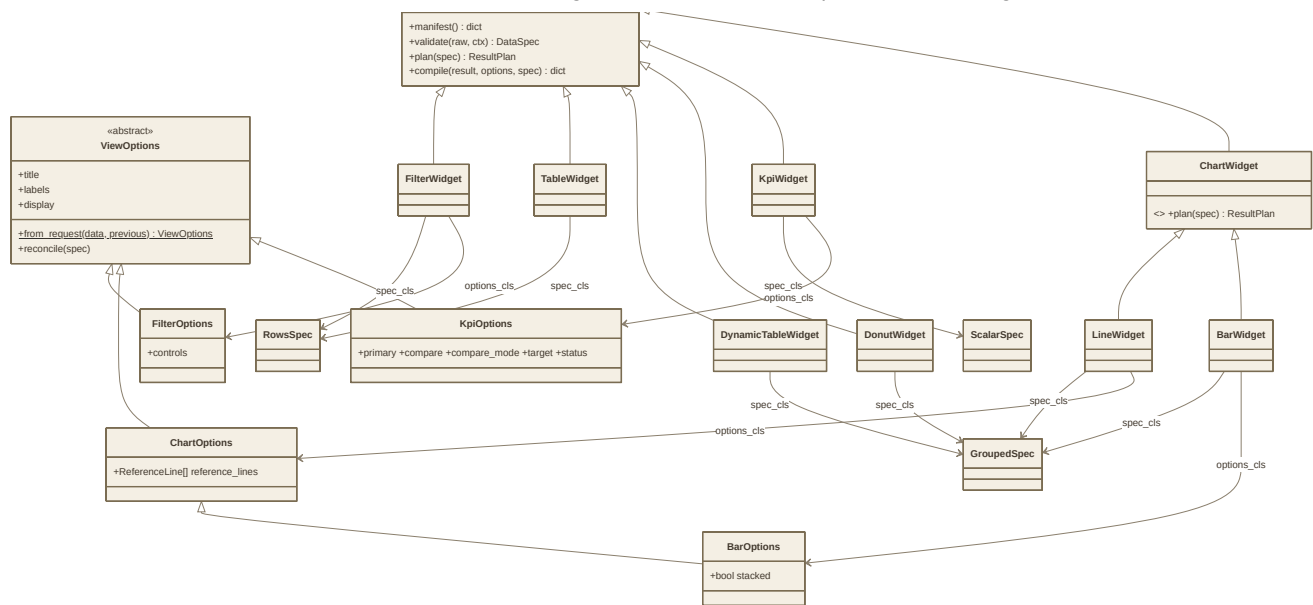
Prioridad por banda: 0 BUSINESS (PivotMultiMetric, bloqueante) · 100 STRUCTURE · 200 METRICS · 300 REFERENCES. El orden de los errores lo deciden estas bandas, nunca la posición en rules().

5 · engine/ — Planes y resultados tipados



6 · widgets/ — Tipos de widget y ViewOptions

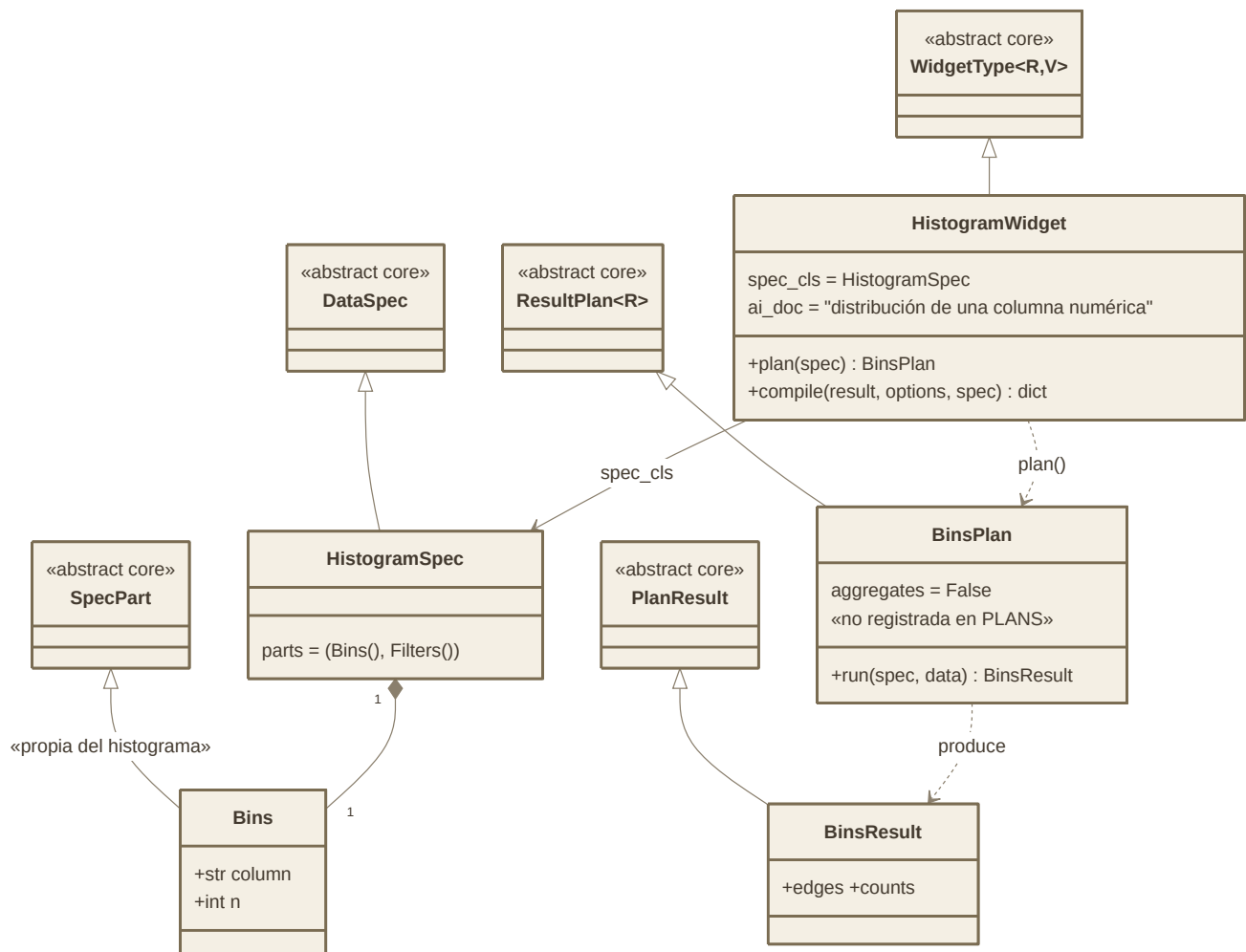




bar y line comparten ChartWidget, que elige flat o pivot_chart según haya pivote — es la única decisión de plan que no es un plan_key fijo.

7 · Caso completo — widget de extensión (histogram)

Todas las piezas de arriba, encajando en el ejemplo de tu propio documento: un widget que vive en un solo archivo de widgets/ext/, con una forma de dato y un plan nuevos, sin tocar nada del core.



Diferencia clave con los widgets del core: BinsPlan no se registra en PLANS porque hoy solo lo usa este widget — plan() lo devuelve directo. El día que un segundo widget lo necesite, se promueve a

engine/plans/ , *siguiendo la misma regla que ya discutimos para rows* .